C Programming Language

Complete Core & Advanced Reference Manual

Document Type: Comprehensive Study Notes & Technical Reference

Target Audience: Beginners to Intermediate Software Developers

Language Standard: ANSI C / C99 / C11

Coverage: Syntax, Memory Management, Pointers, Structures, Data Structures & Best Practices

Table of Contents

1. Introduction to C & Environment Setup 2

2. Basic Syntax, Program Structure & Input/ Output 3

3. Data Types, Variables & Operators 4

4. Control Flow Statements (Conditionals & Loops) 5

6

5. Functions, Scope & Recursion
6. Arrays & Strings 7
7. Pointers & Memory Addressing 8
8. Dynamic Memory Allocation (Heap Management) 9
9. Structures, Unions & Bit-Fields 10
10. File Handling & I/O Operations 11
11. Preprocessor Directives, Header Files & Best Practices 12

1. Introduction to C & Environment Setup

C is a procedural, general-purpose programming language developed by **Dennis Ritchie** between 1969 and 1973 at Bell Labs. It was designed to construct the Unix operating system and has become one of the most widely used systems programming languages of all time.

Key Features of C

- **Efficiency and Speed:** Compiles directly to machine-level code, offering low-level memory access and execution speed second only to Assembly.
- **Portability:** C programs can be compiled across different processor architectures with minimal modifications.
- **Structured Programming:** Supports modular programming using functions, code blocks, and scope controls.
- **Rich Library Support:** Includes extensive built-in standard library functions (C Standard Library).

The Compilation Process

Converting C source code (`.c`) into an executable binary involves four distinct phases carried out by the build toolchain:

Phase	Input File	Output File	Description
1. Preprocessing	`main.c`	`main.i`	Expands macros, includes header files (`#include`), removes comments, and handles conditional compilation.
2. Compilation	`main.i`	`main.s`	Translates preprocessed source code into hardware-specific assembly instructions.
3. Assembly	`main.s`	`main.o` / `main.obj`	Converts assembly code into relocatable binary machine code (object files).
4. Linking	`main.o`	`main.exe` / `a.out`	Combines object files with system libraries to resolve function calls and build the final executable.

Setting Up GCC Compiler

To compile C programs using the GNU Compiler Collection (GCC):

```
# Compile a source file into an executable named output_app
gcc -Wall -Wextra -O2 main.c -o output_app

# Run the compiled application
./output_app
```

> **Best Practice Tip**
>
> Always compile with `-Wall -Wextra` flags enabled. This forces GCC to emit warnings about potential runtime issues, uninitialized variables, and type mismatches.

2. Basic Syntax, Program Structure & Input/Output

Every standard C application must contain a entry point function named `main()`. The following program demonstrates foundational elements including headers, variables, output formatting, and user input.

Anatomy of a C Program

```c
/* Standard I/O library header inclusion */
#include <stdio.0h>
#include <stdlib.h>

/* Application Entry Point */
int main(void) {
    // Declare and initialize variables
    int age = 25;
    double weight = 70.5;

    // Formatted Output
    printf("User Age: %d years
", age);
    printf("User Weight: %.2f kg
", weight);

    return 0; // Status code 0 indicates success
}
```

Formatted I/O: `printf` and `scanf`

Standard input/output is provided by `<stdio.h>`. Modifiers control precision, field width, and data conversion.

Format Specifier	Data Type	Example Output / Usage
`%d` / `%i`	Signed Integer	`printf("%d", 42);` → `42`
`%f`	Floating-point (Float)	`printf("%.2f", 3.14159);` → `3.14`
`%lf`	Double-precision Float	`scanf("%lf", &doubleVal);`
`%c`	Single Character	`printf("%c", 'A');` → `A`
`%s`	String (Character Array)	`printf("%s", "Hello");` → `Hello`
`%p`	Pointer Address (Hexadecimal)	`printf("%p", (void*)&var);` → `0x7ffd56a8`
`%zu`	`size_t` Type	`printf("%zu", sizeof(int));` → `4`

Critical Security Warning: `scanf` Buffer Overflow

When reading strings using `scanf("%s", buffer)`, unbounded input can cause buffer overflows. Always specify field width, e.g., `scanf("%19s", buffer);` for a 20-character array, or use `fgets()`.

3. Data Types, Variables & Operators

C is a statically-typed, strongly-typed language. Variables must be declared before use, defining the size and memory layout of stored values.

Fundamental Primitive Types

Type Group	Keyword	Typical Size (32/64-bit)	Value Range
Character	`char`	1 Byte (8 bits)	-128 to 127 or 0 to 255
Integer	`int`	4 Bytes (32 bits)	-2,147,483,648 to 2,147,483,647
Short Int	`short`	2 Bytes (16 bits)	-32,768 to 32,767
Long Int	`long`	4 or 8 Bytes	-2^31 to 2^31-1 (or -2^63 to 2^63-1)
Float	`float`	4 Bytes	~1.2E-38 to ~3.4E+38 (6 decimal precision)
Double	`double`	8 Bytes	~2.3E-308 to ~1.7E+308 (15 decimal precision)

Operators in C

C includes built-in operators categorized by function:

1. Arithmetic Operators

`+` (Addition), `-` (Subtraction), `*` (Multiplication), `/` (Division), `%` (Modulo - integer remainder only).

2. Bitwise Operators

Bitwise operators manipulate individual bits within integer types directly:

```
unsigned char a = 0b00001100; // 12 in decimal
unsigned char b = 0b00011001; // 25 in decimal

unsigned char ans_and = a & b;  // Bitwise AND : 00001000 (8)
unsigned char ans_or  = a | b;  // Bitwise OR  : 00011101 (29)
unsigned char ans_xor = a ^ b;  // Bitwise XOR : 00010101 (21)
unsigned char ans_not = ~a;     // Bitwise NOT : 11110011 (243)
unsigned char ans_shift = a << 2; // Left Shift  : 00110000 (48)
```

4. Control Flow Statements

Control flow directives determine execution paths based on boolean evaluations and iterative loops.

Conditional Execution

```c
int score = 85;

if (score >= 90) {
    printf("Grade A
");
} else if (score >= 80) {
    printf("Grade B
");
} else {
    printf("Grade C or lower
");
}

/* Switch statement for discrete integer evaluation */
char flag = 'Y';
switch(flag) {
    case 'Y':
    case 'y':
        printf("Action Approved
");
        break;
    case 'N':
        printf("Action Denied
");
        break;
    default:
        printf("Invalid Choice
");
        break;
}
```

Looping Constructs

C provides three primary loop structures: `for`, `while`, and `do-while`.

```c
// 1. For Loop - Defined Iteration Count
for (int i = 0; i < 5; i++) {
    printf("Iteration: %d
", i);
}

// 2. While Loop - Condition Checked First
int count = 5;
while (count > 0) {
    printf("Countdown: %d
", count);
    count--;
}

// 3. Do-While Loop - Executes at least once
int input_val;
do {
    printf("Enter a positive number: ");
    scanf("%d", &input_val);
} while (input_val <= 0);
```

5. Functions, Scope & Recursion

Functions allow modular code organization, promoting code reuse, isolation, and easier debugging.

Function Declaration vs Definition

```c
#include <stdio.h>

// Function Prototype / Declaration
int calculateFactorial(int n);

int main(void) {
    int num = 5;
    int result = calculateFactorial(num);
    printf("Factorial of %d is %d
", num, result);
    return 0;
}

// Function Definition
int calculateFactorial(int n) {
    if (n <= 1) return 1; // Base case
    return n * calculateFactorial(n - 1); // Recursive step
}
```

Parameter Passing: Value vs Reference

- **Pass by Value:** Creates a copy of the passed variable. Changes inside the function do not affect the original variable.
- **Pass by Reference:** Passes variable memory addresses using pointers, allowing direct mutation of original variables.

```c
// Pass by Value
void modifyVal(int x) { x = 100; }

// Pass by Reference (via Pointers)
void modifyRef(*int x_ptr) { *x_ptr = 100; }
```

6. Arrays & Strings

An array is a contiguous block of memory storing elements of identical data types.

Single and Multidimensional Arrays

```c
// 1D Array Declaration & Initialization
int numbers[5] = {10, 20, 30, 40, 50};

// Accessing Elements (0-indexed)
int thirdElement = numbers[2]; // 30

// 2D Matrix Array (3 rows, 3 columns)
int matrix[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

Strings in C (Character Arrays)

In C, strings are null-terminated character arrays terminated by the `''` null character.

```c
#include <string.h>

char str1[20] = "Hello";
char str2[] = "World";

// String manipulations from <string.h>
size_t len = strlen(str1);        // Returns 5
strcat(str1, str2);              // Appends str2 to str1 ("HelloWorld")
strcpy(str1, "Reset");            // Copies "Reset" into str1
int cmp = strcmp(str1, "Reset");  // Returns 0 if matching
```

7. Pointers & Memory Addressing

Pointers are variables that store physical memory addresses of other variables. They enable direct hardware access, fast array processing, and dynamic allocation.

Pointer Syntax and Operations

```c
int target = 42;
int *ptr = &target; // ptr stores address of target (& = address-of operator)

printf("Value of target: %d
", target);     // Output: 42
printf("Address of target: %p
", (void*)&target);
printf("Stored in ptr: %p
", (void*)ptr);
printf("Dereferenced *ptr: %d
", *ptr);     // Output: 42 (* = dereference operator)

// Modifying underlying data via pointer
*ptr = 99;
printf("New value of target: %d
", target); // Output: 99
```

Pointer Arithmetic & Arrays

Array names act as constant pointers to their first element (`array == &array[0]`).

```c
int arr[3] = {100, 200, 300};
int *p = arr;

// Increments pointer address by sizeof(int) bytes
printf("Element 0: %d
", *p);       // 100
printf("Element 1: %d
", *(p + 1)); // 200
printf("Element 2: %d
", *(p + 2)); // 300
```

> **Null & Wild Pointers**
>
> Uninitialized pointers contain garbage addresses (Wild Pointers). Always initialize unused pointers to `NULL` : `int *ptr = NULL;` . Dereferencing a NULL pointer leads to a Segmentation Fault.

8. Dynamic Memory Allocation (Heap Management)

Dynamic memory allocation allows programs to request memory from the operating system at runtime (Heap memory) when array dimensions are unknown at compile time.

Memory Management Functions (`<stdlib.h>`)

Function	Signature	Description
`malloc`	`void* malloc(size_t size)`	Allocates specified bytes of unitialized memory. Returns `NULL` on failure.
`calloc`	`void* calloc(size_t num, size_t size)`	Allocates memory for array elements and zero-initializes all bits.
`realloc`	`void* realloc(void* ptr, size_t new_size)`	Resizes previously allocated memory block without losing data.
`free`	`void free(void* ptr)`	Deallocates heap memory returning it back to system pools.

Complete Heap Allocation Example

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;
    int *arr = (int*)malloc(n * sizeof(int));

    // Verify allocation success
    if (arr == NULL) {
        fprintf(stderr, "Memory Allocation Failed!
");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        arr[i] = (i + 1) * 10;
    }

    // Free allocated memory block to prevent leaks
    free(arr);
    arr = NULL; // Avoid dangling pointer
    return 0;
}
```

9. Structures, Unions & Bit-Fields

Structures (`struct`) group variables of different types under a single unified name.

Structures Definition and Padding

```c
// Struct Definition
struct Student {
    int id;
    char name[50];
    float gpa;
};

// Usage with Typedef for clean alias instantiation
typedef struct {
    double x;
    double y;
} Point;

int main(void) {
    struct Student s1 = {101, "Alice", 3.9};
    Point p1 = {12.5, 45.0};

    // Accessing members via dot operator
    printf("Student: %s, GPA: %.2f
", s1.name, s1.gpa);

    // Accessing structure pointers using arrow operator (->)
    struct Student *ptr = &s1;
    printf("ID: %d
", ptr->id);

    return 0;
}
```

Unions vs Structures

Unlike structures, members of a **Union** share the same memory location. A union size equals the size of its largest member variable.

```c
union DataPayload {
    int intVal;
    float floatVal;
    char strVal[20];
}; // Memory allocated is 20 bytes (largest member size)
```

10. File Handling & I/O Operations

The standard C library provides stream interfaces to interact with disks and external media via `FILE` pointers.

File Operations Lifecycle

1. Open a stream using `fopen()` with appropriate mode (`"r"` , `"w"` , `"a"` , `"rb"` , `"wb"`).
2. Perform IO read/write operations using `fprintf` , `fscanf` , `fputs` , `fgets` , `fread` , or `fwrite` .
3. Close the active stream using `fclose()` to flush buffers.

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("log.txt", "w");
    if (fp == NULL) {
        perror("Error opening file");
        return 1;
    }

    // Writing text to file
    fprintf(fp, "System Log Entry: %s
", "Operation Successful");
    fclose(fp);

    // Reading text back from file
    char buffer[100];
    fp = fopen("log.txt", "r");
    if (fp && fgets(buffer, sizeof(buffer), fp)) {
        printf("File Contents: %s", buffer);
    }
    fclose(fp);

    return 0;
}
```

11. Preprocessor Directives & Best Practices

The C preprocessor modifies code before compilation occurs.

Common Directives & Macro Safeguards

```c
// 1. Header Guard (Prevents Multiple Inclusion Errors)
#ifndef MY_HEADER_H
#define MY_HEADER_H

// 2. Macro Definitions with Parenthesis Safety
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))

// 3. Conditional Compilation
#ifdef DEBUG
    #define LOG(msg) printf("DEBUG: %s
", msg)
#else
    #define LOG(msg)
#endif

#endif // MY_HEADER_H
```

Summary of Essential C Programming Best Practices

- Always initialize variables before reading them to avoid unpredictable values.
- Check return values of memory routines (`malloc` , `calloc`) and file operations (`fopen`).
- Match every `malloc()` call with a corresponding `free()` to prevent memory leaks.
- Use `const` for parameters that should not be modified by function implementations.
- Always protect preprocessor multi-statement macros using proper parenthesization.